

Software Requirements Specification (SRS)

Unified Log Analyzer & Correlation Engine (ULACE)

A comprehensive system specification for multi-source security log aggregation, parsing, real-time visual correlation, and localized data retention management.

Document Version:	v1.0.0
Date:	June 8, 2026
Status:	Draft / Initial Specification
Target Audience:	Engineering Team, Frontend UI/UX Designers, Database Architects

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) documents the complete functional, non-functional, and technical requirements for the **Unified Log Analyzer & Correlation Engine (ULACE)**. This system is engineered to ingest high-volume log data from disparate network and security infrastructure sources—including SIEM, Network Detection and Response (NDR), and Cisco Stealthwatch (Secure Network Analytics). It parses, standardizes, correlates, caches dynamically based on strict data retention limits, and translates complex threat data into interactive graphical visualizations.

1.2 Scope

ULACE provides security analysts with a lightweight, visual interface to inspect network activity and trace individual IP interactions across heterogeneous logs. The system components comprise:

- **Ingestion Component:** An extensible ingestion layer beginning with local mock CSV adapters and progressing to live log streaming (Syslog/API).
- **Parsing & Normalization Layer:** An internal pipeline to extract parameters (IPs, ports, protocols, timestamps, actions) into a unified JSON format.
- **Data Management Layer:** A time-delimited relational or time-series data store operating a rolling time-window pruning loop to enforce storage bounds.
- **Visualization Dashboard:** A web front-end containing cross-reference network graph maps, time-series volume trend lines, and contextual drill-down views.

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
SIEM	Security Information and Event Management system aggregating enterprise logs.
NDR	Network Detection and Response platform focusing on network-layer malicious patterns.
Stealthwatch	Cisco Secure Network Analytics; utilizes NetFlow telemetry to observe internal host traffic.
Data Pruning	Automated background script purging database records older than a configured timestamp boundary to guarantee stable infrastructure overhead.
IP Correlation	The systemic link discovery mapping source and destination IPs across unrelated security systems.

2. Overall Description

2.1 Product Perspective

ULACE functions as an intermediate analytical framework situated between raw enterprise logging endpoints and a frontend presentation layer. Rather than permanently archiving massive logs long-term, it offers high-speed, localized analysis over a rolling temporal cache. The initial design begins with standard CSV ingest schemas mimicking live systems to simplify parser verification before physical hookups take place.

2.2 Product Functions

The primary core services performed by the system include:

1. **CSV Ingestion & Simulation:** Simulates stream data reading from structured multi-source security CSV files containing timestamped events.
2. **Multi-Schema Normalized Parsing:** Matches unique input formats (SIEM alert strings, NDR telemetry blocks, Stealthwatch flow records) and unifies fields into standard keys.
3. **Bounded Database Management:** Keeps records active only within user-defined historical thresholds (e.g., past 24 hours, 7 days), transparently recycling storage blocks.
4. **Relational Context Querying:** Fetches interconnected transactions involving specific IPs across different source logs.

5. **Dynamic Interactive Dashboarding:** Visualizes indicators through charts showing event spikes, distribution metrics, and host nodes.

2.3 User Classes and Characteristics

- **SOC Security Analyst:** Looks up suspicious internal or external IPs, views graphical relationships, traces timelines, and discovers lateral movement or scanning anomalies.
- **System Administrator / Engineer:** Configures operational constants, updates log parsers, checks parsing errors, and adjusts sliding window expiration bounds.

2.4 Assumptions and Dependencies

- **Log Timestamps:** Assumes all input files/streams supply standardized ISO-8601 or Epoch UTC formats, or contain clear time zones to allow accurate chronological sorting.
- **Local Ingestion Phase:** The initial builds depend entirely on valid structured local CSV formats containing an identifier designating the pseudo-source system.

3. External Interface Requirements

3.1 User Interfaces

The web UI must prioritize dense analytical data representation using a responsive design framework. Key operational modules comprise:

- **Main Dashboard Workspace:** Provides quick-glance distribution summaries, live count cards, and high-level categorization views.
- **IP Interaction Explorer (Graph Layout):** A node-link map showing communication strings where circles represent distinct IPs and links represent transaction frequency or hazard weights.
- **Log Tabular Drill-Down View:** Paginated query sheets where selecting specific node points filters log contents instantly.

3.2 Software Interfaces

Component Link	Interface Specification
Log File Interface	Local file system input streams utilizing robust CSV read buffers (e.g., Python Pandas / native stream utilities).
Database Connection	SQL / Time-Series API wrapper utilizing connection pools to insert parsed data points quickly.
Frontend API	RESTful JSON endpoints exposing queries for timeline metrics, IP connectivity arrays, and node details.

4. System Features

4.1 Feature: Ingestion & Parsing Engine (Initial Stage)

4.1.1 Description and Priority

Converts unstructured/semi-structured file entries into standardized objects. **Priority: High.** This asset underpins all database schemas and visualization components.

4.1.2 Functional Requirements

- **REQ-ING-1:** The parser must ingest a unified or distinct CSV layout mapping `source_system` (SIEM, NDR, Stealthwatch), `timestamp`, `source_ip`, `destination_ip`, `source_port`, `destination_port`, `protocol`, `alert_severity`, and `message_payload`.
- **REQ-ING-2:** Validations must catch missing IP parameters or malformed entries and safely route errors to a diagnostic dump without interrupting ongoing batch reads.

[Sample Reference Input Mock Format]

```
timestamp,source_system,source_ip,destination_ip,src_port,dest_port,protocol,severity,message
2026-06-08T22:00:00Z,STEALTHWATCH,192.168.1.50,10.0.0.12,49152,443,TCP,LOW,High flow
volume detected
2026-06-08T22:01:05Z,SIEM,10.0.0.12,192.168.1.105,80,51234,TCP,HIGH,Brute force
signature matched
2026-06-08T22:02:12Z,NDR,192.168.1.105,8.8.8.8,55200,53,UDP,MEDIUM,DNS Tunneling
anomaly suspected
```

4.2 Feature: Bounded Data Retention (Sliding Time Window)

4.2.1 Description and Priority

Maintains storage stability by maintaining a restricted data pool window. **Priority: High.** Without automated purging, large enterprise telemetry would swamp standard disk infrastructure rapidly.

4.2.2 Functional Requirements

- **REQ-RET-1:** The storage module must expose a parameter configuration value (e.g., `RETENTION_WINDOW_HOURS = 24`).
- **REQ-RET-2:** A background daemon must execute sequentially every 5 minutes, evaluating entries via:
$$\text{Timestamp_Current} - \text{Timestamp_Record} > \text{Retention_Threshold}$$
Any items evaluating to true must be removed using transactional block deletions.

4.3 Feature: Cross-Log IP Activity & Connection Querying

4.3.1 Description and Priority

Aggregates transactions across all platforms relating to specific IP entities. **Priority: High.**

4.3.2 Functional Requirements

- **REQ-QRY-1:** The endpoint layer must accept a query string parameter containing a specific target IP string.
- **REQ-QRY-2:** The return output must fetch instances where the requested target IP matches either the `source_ip` or `destination_ip` across SIEM, NDR, and Stealthwatch collections simultaneously within the database.

4.4 Feature: Interactive Frontend Visualizations

4.4.1 Description and Priority

Provides interactive dashboards to replace textual data parsing with actionable visual layouts. **Priority: Medium.**

4.4.2 Functional Requirements

- **REQ-VIS-1 (Network Topology Map):** Must chart explicit structural linkages between interacting node IPs, allowing analysts to hover over linkages to inspect active alerts or specific ports.
- **REQ-VIS-2 (Multi-Source Volume Chart):** An stacked line or bar visualization sorting log entries on an active chronological axis, color-coded by the source system to highlight coordinated security events.
- **REQ-VIS-3 (Severity Distribution Matrix):** Pie/Donut charts profiling active counts of low, medium, and high severity items within the current database retention period.

5. Non-Functional Requirements

5.1 Performance Requirements

- **Parser Processing Velocity:** The parsing pipeline should process at least 5,000 raw log lines per second from buffered CSV blocks on baseline runtime targets.
- **Frontend API Response Limits:** Data aggregation calls for any specified IP node spanning up to 50,000 internal logs must deliver payload packets within less than 1.5 seconds.

5.2 Security Requirements

- **Data Injection Guarding:** Log parsing modules must process incoming properties strictly as descriptive data properties rather than interpretable code strings, eliminating script injection bugs.
- **UI Execution Sandboxing:** Frontend chart widgets must sanitize message text blocks to block cross-site scripting (XSS) risks inside tooltip windows.

5.3 Software Quality Attributes

- **Scalability:** Ingestion pipelines must deploy separated ingestion adapters so transitioning from local CSV reading to functional network sockets requires zero alterations to parsing or database logic.
- **Maintainability:** Modular extraction rules should be isolated inside individual configuration schemas, enabling support for brand new infrastructure sources without modifying core code frameworks.